CSA4704 - DEEP LEARNING
Lab Experiments

Name: Harini. R
Register no: 192324108

Experiment 26: Gibbs Sampling and Contrastive Divergence Analysis
Problem Statement: The streaming team finds that its RBM learns slowly and poorly. The research lead asks how many Gibbs steps are worth the extra computation. The binarised MNIST or user–movie data from Experiment 25. The objective is to analyse Gibbs sampling chains and compare CD-1, CD-k and Persistent CD. Use NumPy, SciPy, Matplotlib.
Tasks:
1. Implement block Gibbs sampling between visible and hidden layers.
2. Run chains from random noise and display samples after 1, 10, 100 and 1000 steps.
3. Train RBMs with CD-1, CD-5, CD-10 and PCD.
4. Plot reconstruction error and pseudo-log-likelihood for each method.
5. Compare training time against sample quality.

Formulas:

1. Gibbs sampling

$$h^{(t)} \sim P(h|v^{(t)})$$

$$v^{(t+1)} \sim P(v|h^{(t)})$$

2. CD-k

$$CD_k = E_0[E(v,h)] - E_k[E(v,h)]$$

This is a shorthand for the energy difference used in the CD objective; the RBM gradient is estimated using the positive and negative phase statistics.

3. Reconstruction error

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(v_i - \hat{v}_i)^2$$

4. Free energy

$$F(v) = -a^T v - \sum_j \log\left(1 + e^{b_j + W_j^T v}\right)$$

Code:

```
import time
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist

np.random.seed(42)

# 1. Load binarized MNIST
(x_train, _), (x_test, _) = mnist.load_data()

x_train = (x_train[:5000].reshape(-1, 784) / 255.0 > 0.5).astype(np.float32)
x_test = (x_test[:500].reshape(-1, 784) / 255.0 > 0.5).astype(np.float32)

N_VISIBLE = 784
N_HIDDEN = 64
BATCH_SIZE = 64
EPOCHS = 5
LR = 0.01

def sigmoid(x):
    x = np.clip(x, -30, 30)
    return 1.0 / (1.0 + np.exp(-x))

class RBM:
    def __init__(self, n_visible=784, n_hidden=64, seed=42):
        rng = np.random.default_rng(seed)
        self.rng = rng
        self.W = rng.normal(0, 0.01, (n_visible, n_hidden)).astype(np.float32)
        self.a = np.zeros(n_visible, dtype=np.float32)
        self.b = np.zeros(n_hidden, dtype=np.float32)

    def p_hidden(self, v):
        return sigmoid(v @ self.W + self.b)

    def p_visible(self, h):
        return sigmoid(h @ self.W.T + self.a)
```

```
def sample_bernoulli(self, probs):
    return (self.rng.random(probs.shape) < probs).astype(np.float32)

def gibbs_step(self, v):
    ph = self.p_hidden(v)
    h = self.sample_bernoulli(ph)
    pv = self.p_visible(h)
    v_new = self.sample_bernoulli(pv)
    return v_new, pv

def free_energy(self, v):
    hidden_input = v @ self.W + self.b
    return -np.sum(v * self.a, axis=1) - np.sum(
        np.logaddexp(0, hidden_input), axis=1
    )

def pseudo_log_likelihood(self, v):
    # Approximate pseudo-log-likelihood by flipping one random bit
    # per sample and comparing free energies.
    v = v.copy()
    rows = np.arange(len(v))
    cols = self.rng.integers(0, v.shape[1], size=len(v))
    original_bits = v[rows, cols].copy()
    v[rows, cols] = 1 - original_bits

    fe_original = self.free_energy(v.copy())
    v[rows, cols] = original_bits
    fe_flipped = self.free_energy(v)

    # A commonly used stochastic estimate of pseudo-log-likelihood
    bit_log_prob = -np.logaddexp(0, fe_flipped - fe_original)
    return np.mean(v.shape[1] * bit_log_prob)

def train_batch(self, v0, method="CD", k=1, persistent=None, lr=0.01):
    ph0 = self.p_hidden(v0)

    if method == "PCD" and persistent is not None:
        vk = persistent.copy()
    else:
        vk = v0.copy()

    for _ in range(k):
        vk, _ = self.gibbs_step(vk)
```

```
        phk = self.p_hidden(vk)

        # CD/PCD approximate gradient updates
        self.W += lr * ((v0.T @ ph0) - (vk.T @ phk)) / len(v0)
        self.a += lr * np.mean(v0 - vk, axis=0)
        self.b += lr * np.mean(ph0 - phk, axis=0)

        return vk

# 2. Demonstrate Gibbs chains from random binary noise
demo_rbm = RBM(seed=123)
v_chain = (np.random.rand(10, 784) > 0.5).astype(np.float32)
snapshots = {0: v_chain.copy()}

for step in range(1, 1001):
    v_chain, _ = demo_rbm.gibbs_step(v_chain)
    if step in [1, 10, 100, 1000]:
        snapshots[step] = v_chain.copy()

fig, axes = plt.subplots(4, 10, figsize=(12, 5))
for row, step in enumerate([1, 10, 100, 1000]):
    for col in range(10):
        axes[row, col].imshow(snapshots[step][col].reshape(28, 28), cmap="gray")
        axes[row, col].axis("off")
        if col == 0:
            axes[row, col].set_ylabel(f"{step} steps")
plt.suptitle("Gibbs Sampling Chain Snapshots (Untrained RBM)")
plt.tight_layout()
plt.show()

# 3. Train CD-1, CD-5, CD-10 and PCD
settings = [
    ("CD-1", "CD", 1),
    ("CD-5", "CD", 5),
    ("CD-10", "CD", 10),
    ("PCD", "PCD", 1)
]

results = {}
histories = {}

for name, method, k in settings:
    print(f"Training {name}...")
```

```
    rbm = RBM(seed=42)
    persistent = None
    if method == "PCD":
        persistent = (np.random.rand(BATCH_SIZE, N_VISIBLE) > 0.5).astype(np.float32)

    hist = {"reconstruction_error": [], "pll": []}
    start_time = time.time()

    for epoch in range(EPOCHS):
        order = np.random.permutation(len(x_train))
        batch_errors = []

        for start in range(0, len(x_train), BATCH_SIZE):
            batch = x_train[order[start:start + BATCH_SIZE]]

            if method == "PCD":
                if len(batch) != BATCH_SIZE:
                    continue
                persistent = rbm.train_batch(
                    batch, method="PCD", k=1,
                    persistent=persistent, lr=LR
                )
            else:
                rbm.train_batch(batch, method="CD", k=k, lr=LR)

            _, pv = rbm.gibbs_step(batch)
            batch_errors.append(np.mean((batch - pv) ** 2))

        hist["reconstruction_error"].append(np.mean(batch_errors))
        hist["pll"].append(rbm.pseudo_log_likelihood(x_test[:100]))

    elapsed = time.time() - start_time
    results[name] = {"rbm": rbm, "time": elapsed}
    histories[name] = hist

    print(f"{name}: {elapsed:.1f} seconds")

# 4. Plot reconstruction error
plt.figure(figsize=(9, 5))
for name, hist in histories.items():
    plt.plot(hist["reconstruction_error"], marker="o", label=name)
plt.title("Reconstruction Error by Training Method")
plt.xlabel("Epoch")
plt.ylabel("Reconstruction MSE")
```

```
plt.legend()
plt.grid(True)
plt.show()

# 5. Plot pseudo-log-likelihood
plt.figure(figsize=(9, 5))
for name, hist in histories.items():
    plt.plot(hist["pll"], marker="o", label=name)
plt.title("Pseudo-Log-Likelihood by Training Method")
plt.xlabel("Epoch")
plt.ylabel("Approximate pseudo-log-likelihood")
plt.legend()
plt.grid(True)
plt.show()

# 6. Compare time and final metrics
summary = []
for name, hist in histories.items():
    summary.append({
        "Method": name,
        "Training Time (s)": round(results[name]["time"], 2),
        "Final Reconstruction MSE": round(hist["reconstruction_error"][-1], 4),
        "Final Pseudo-Log-Likelihood": round(hist["pll"][-1], 2)
    })

import pandas as pd
summary_df = pd.DataFrame(summary)
display(summary_df)

# Display reconstructions from each trained model
test_batch = x_test[:10]
fig, axes = plt.subplots(len(settings) + 1, 10, figsize=(13, 7))

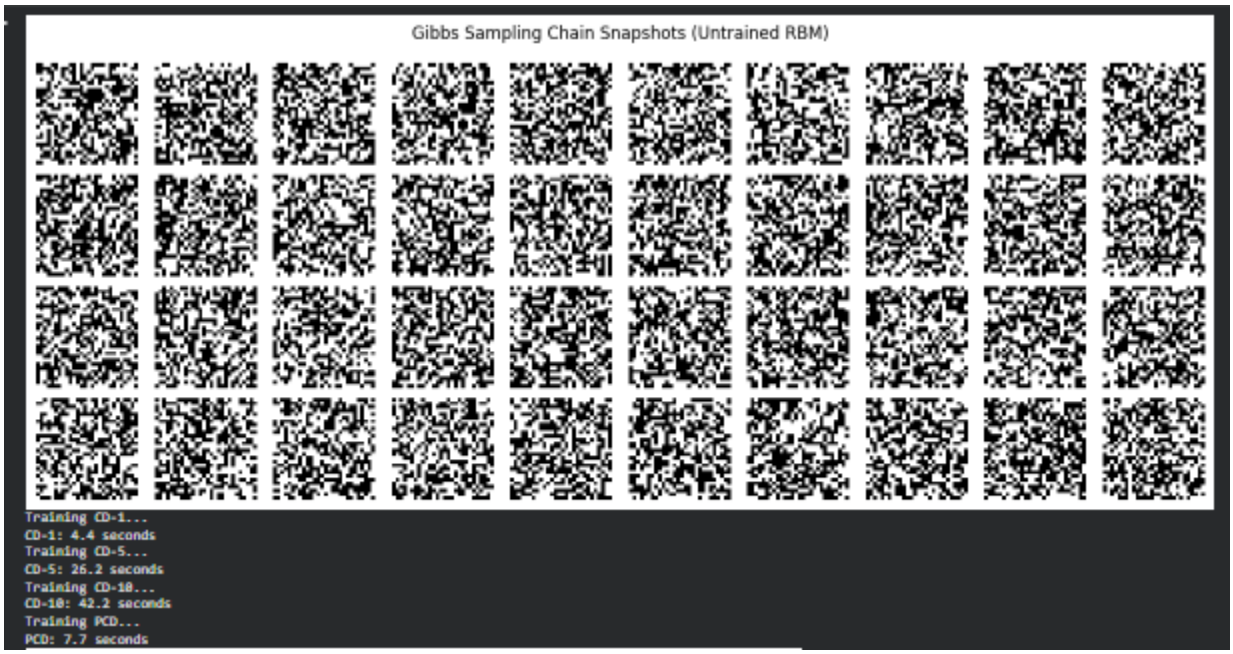
for i in range(10):
    axes[0, i].imshow(test_batch[i].reshape(28, 28), cmap="gray")
    axes[0, i].axis("off")
    if i == 0:
        axes[0, i].set_ylabel("Original")

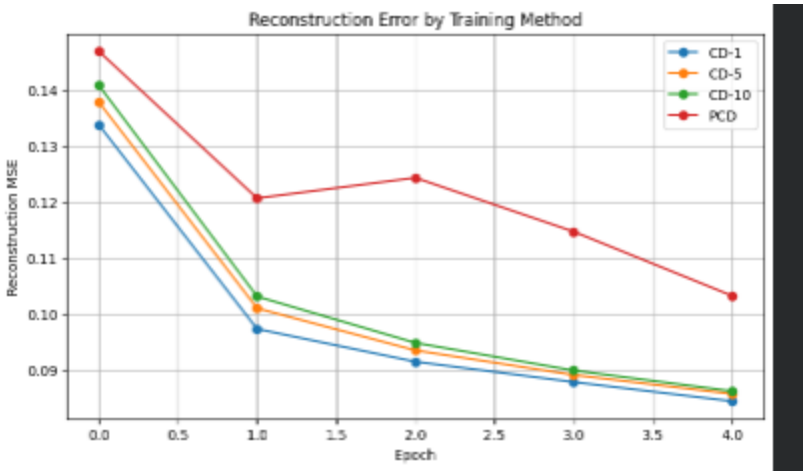
for row, (name, _, _) in enumerate(settings, start=1):
    rbm = results[name]["rbm"]
    probs = rbm.p_visible(rbm.p_hidden(test_batch))
    for col in range(10):
        axes[row, col].imshow(probs[col].reshape(28, 28), cmap="gray")
```

```
        axes[row, col].axis("off")
        if col == 0:
            axes[row, col].set_ylabel(name)

plt.suptitle("RBM Reconstructions by Training Method")
plt.tight_layout()
plt.show()
```

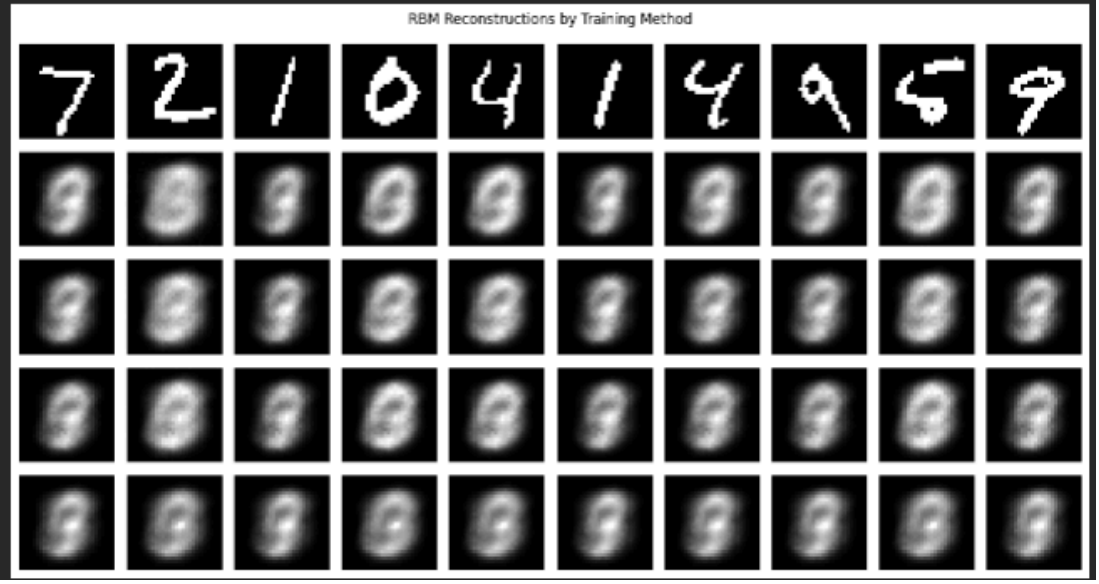
Output:







	Method	Training Time (s)	Final Reconstruction MSE	Final Pseudo-Log-Likelihood
0	CD-1	4.40	0.0844	-179.199997
1	CD-5	26.25	0.0857	-214.399994
2	CD-10	42.23	0.0862	-242.500000
3	PCD	7.72	0.1032	-183.800003



Experiment 27: Deep Belief Network Representation Learning
Problem Statement: A postal service has very few labelled handwritten PIN-code digits but many unlabelled ones. It wants to pre-train features on the unlabelled data first. MNIST digits, using only 1,000 labels for fine-tuning. The objective is to build a DBN by stacking RBMs, pre-train greedily and fine- tune for classification. Use NumPy, SciPy, TensorFlow, Keras, Matplotlib.
Tasks:
1. Train two or three stacked RBMs layer by layer (784–500–250).
2. Visualise the learned weights of the first layer as image filters.
3. Transfer the weights into a Keras dense network and fine-tune with the labelled data.
4. Train the same network from random initialisation for comparison.
5. Compare accuracy and plot learned representations in 2-D.

Formulas:

1. First RBM hidden representation

$$h^{(1)} = \sigma(W_1^T x + b_1)$$

2. Second RBM hidden representation

$$h^{(2)} = \sigma(W_2^T h^{(1)} + b_2)$$

3. Fine-tuning output

\[\hat y=\operatorname{softmax}(W_oh^{(2)}+b_o) \]

4. Cross-entropy loss

$$L = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

5. Classification accuracy

$$Accuracy = \frac{Correctpredictions}{Totalpredictions} \times 100$$

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
```

```
import time

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.utils import to_categorical
from sklearn.decomposition import PCA

# Reproducibility
np.random.seed(42)
tf.random.set_seed(42)

# ----------------------------------------------------------
# 1. Load and preprocess MNIST
# ----------------------------------------------------------

(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Flatten images from 28x28 to 784 features
x_train = x_train.reshape(-1, 784).astype("float32") / 255.0
x_test = x_test.reshape(-1, 784).astype("float32") / 255.0

y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)

# Use a smaller subset for faster unsupervised pretraining
x_pretrain = x_train[:5000]

print("Training data shape:", x_train.shape)
print("Test data shape:", x_test.shape)

# ----------------------------------------------------------
# 2. Define an RBM using NumPy
# ----------------------------------------------------------

class RBM:
    def __init__(self, n_visible, n_hidden, learning_rate=0.01):
        self.n_visible = n_visible
        self.n_hidden = n_hidden
        self.learning_rate = learning_rate

        self.W = np.random.normal(
            0, 0.01, (n_visible, n_hidden)
        ).astype(np.float32)
```

```
    self.v_bias = np.zeros(n_visible, dtype=np.float32)
    self.h_bias = np.zeros(n_hidden, dtype=np.float32)

def sigmoid(self, x):
    x = np.clip(x, -50, 50)
    return 1.0 / (1.0 + np.exp(-x))

def sample_hidden(self, visible):
    probabilities = self.sigmoid(
        visible @ self.W + self.h_bias
    )
    samples = (
        np.random.random(probabilities.shape) < probabilities
    ).astype(np.float32)
    return probabilities, samples

def sample_visible(self, hidden):
    probabilities = self.sigmoid(
        hidden @ self.W.T + self.v_bias
    )
    samples = (
        np.random.random(probabilities.shape) < probabilities
    ).astype(np.float32)
    return probabilities, samples

def train(self, data, epochs=3, batch_size=64):
    losses = []

    for epoch in range(epochs):
        indices = np.random.permutation(len(data))
        epoch_loss = 0.0
        batches = 0

        for start in range(0, len(data), batch_size):
            batch = data[indices[start:start + batch_size]]

            # Positive phase
            h0_prob, h0_sample = self.sample_hidden(batch)

            # Negative phase: CD-1
            v1_prob, v1_sample = self.sample_visible(h0_sample)
            h1_prob, _ = self.sample_hidden(v1_sample)
```

```
            # Parameter updates
            batch_n = len(batch)

            positive_grad = batch.T @ h0_prob
            negative_grad = v1_sample.T @ h1_prob

            self.W += self.learning_rate * (
                positive_grad - negative_grad
            ) / batch_n

            self.v_bias += self.learning_rate * np.mean(
                batch - v1_sample, axis=0
            )

            self.h_bias += self.learning_rate * np.mean(
                h0_prob - h1_prob, axis=0
            )

            epoch_loss += np.mean((batch - v1_prob) ** 2)
            batches += 1

        average_loss = epoch_loss / max(batches, 1)
        losses.append(average_loss)

        print(
            f"RBM Epoch {epoch + 1}/{epochs} "
            f"- Reconstruction MSE: {average_loss:.4f}"
        )

    return losses

def transform(self, data):
    probabilities, _ = self.sample_hidden(data)
    return probabilities


# ------------------------------------------------------------
# 3. Pretrain first RBM: 784 -> 500
# ------------------------------------------------------------

print("\nPretraining first RBM...")

rbm1 = RBM(
    n_visible=784,
```

```
    n_hidden=500,
    learning_rate=0.01
)

start_time = time.time()
loss1 = rbm1.train(x_pretrain, epochs=3, batch_size=64)
time1 = time.time() - start_time

# Extract first hidden-layer representations
h1_train = rbm1.transform(x_pretrain)

print("First hidden representation shape:", h1_train.shape)

# ------------------------------------------------------------
# 4. Pretrain second RBM: 500 -> 250
# ------------------------------------------------------------

print("\nPretraining second RBM...")

rbm2 = RBM(
    n_visible=500,
    n_hidden=250,
    learning_rate=0.01
)

start_time = time.time()
loss2 = rbm2.train(h1_train, epochs=3, batch_size=64)
time2 = time.time() - start_time

print("Second RBM pretraining completed.")

# ------------------------------------------------------------
# 5. Build a pretrained neural network
# ------------------------------------------------------------

pretrained_model = Sequential([
    Input(shape=(784,)),
    Dense(
        500,
        activation="sigmoid",
        name="hidden_1"
    ),
    Dense(
        250,
```

```
        activation="sigmoid",
        name="hidden_2"
    ),
    Dense(
        10,
        activation="softmax",
        name="output"
    )
])

# Transfer the RBM weights into the neural network
pretrained_model.get_layer("hidden_1").set_weights([
    rbm1.W,
    rbm1.h_bias
])

pretrained_model.get_layer("hidden_2").set_weights([
    rbm2.W,
    rbm2.h_bias
])

pretrained_model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

# Explicitly call the model to ensure its graph is initialized
_ = pretrained_model(tf.zeros((1, 784)))

# ------------------------------------------------------------
# 6. Fine-tune the pretrained model
# ------------------------------------------------------------

print("\nFine-tuning pretrained model...")

start_time = time.time()

history_pretrained = pretrained_model.fit(
    x_train[:1000],
    y_train_cat[:1000],
    epochs=5,
    batch_size=64,
    validation_split=0.1,
```

```
    verbose=1
)

pretrained_time = time.time() - start_time

pretrained_loss, pretrained_accuracy = pretrained_model.evaluate(
    x_test, y_test_cat, verbose=0
)

# ------------------------------------------------------------
# 7. Build a model with random initialization for comparison
# ------------------------------------------------------------

random_model = Sequential([
    Input(shape=(784,)),
    Dense(500, activation="sigmoid", name="hidden_1"),
    Dense(250, activation="sigmoid", name="hidden_2"),
    Dense(10, activation="softmax", name="output")
])

random_model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

print("\nTraining randomly initialized model...")

start_time = time.time()

history_random = random_model.fit(
    x_train[:1000],
    y_train_cat[:1000],
    epochs=5,
    batch_size=64,
    validation_split=0.1,
    verbose=1
)

random_time = time.time() - start_time

random_loss, random_accuracy = random_model.evaluate(
    x_test, y_test_cat, verbose=0
)
```

```
# ------------------------------------------------------------
# 8. Compare model performance
# ------------------------------------------------------------

print("\n========== MODEL COMPARISON ==========")

print("\nPretrained model:")
print(f"Test loss: {pretrained_loss:.4f}")
print(f"Test accuracy: {pretrained_accuracy * 100:.2f}%")
print(f"Fine-tuning time: {pretrained_time:.2f} seconds")

print("\nRandomly initialized model:")
print(f"Test loss: {random_loss:.4f}")
print(f"Test accuracy: {random_accuracy * 100:.2f}%")
print(f"Training time: {random_time:.2f} seconds")

print("\nRBM pretraining times:")
print(f"First RBM: {time1:.2f} seconds")
print(f"Second RBM: {time2:.2f} seconds")

# ------------------------------------------------------------
# 9. Plot RBM reconstruction losses
# ------------------------------------------------------------

plt.figure(figsize=(8, 5))

plt.plot(
    range(1, len(loss1) + 1),
    loss1,
    marker="o",
    label="RBM 1 (784 -> 500)"
)

plt.plot(
    range(1, len(loss2) + 1),
    loss2,
    marker="s",
    label="RBM 2 (500 -> 250)"
)

plt.xlabel("Epoch")
plt.ylabel("Reconstruction MSE")
plt.title("RBM Pretraining Loss")
```

```
plt.legend()
plt.grid(True)
plt.show()

# ------------------------------------------------------------
# 10. Compare fine-tuning accuracy
# ------------------------------------------------------------

plt.figure(figsize=(8, 5))

plt.plot(
    history_pretrained.history["accuracy"],
    marker="o",
    label="Pretrained model"
)

plt.plot(
    history_random.history["accuracy"],
    marker="s",
    label="Random initialization"
)

plt.xlabel("Epoch")
plt.ylabel("Training Accuracy")
plt.title("Pretrained vs Randomly Initialized Model")
plt.legend()
plt.grid(True)
plt.show()

# ------------------------------------------------------------
# 11. Extract hidden-layer representations
# FIX: use explicit model inputs and initialized graph
# ------------------------------------------------------------

encoder = Model(
    inputs=pretrained_model.inputs,
    outputs=pretrained_model.get_layer("hidden_2").output
)

# Use the first 1000 test examples
test_features = encoder.predict(
    x_test[:1000],
    batch_size=128,
    verbose=0
```

```
)

print("\nExtracted feature shape:", test_features.shape)

# ------------------------------------------------------------
# 12. Visualize representations using PCA
# ------------------------------------------------------------

pca = PCA(n_components=2)
features_2d = pca.fit_transform(test_features)

plt.figure(figsize=(9, 7))

scatter = plt.scatter(
    features_2d[:, 0],
    features_2d[:, 1],
    c=y_test[:1000],
    cmap="tab10",
    s=20,
    alpha=0.75
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA of Learned Hidden Representations")

plt.colorbar(scatter, label="Digit Class")
plt.grid(True)
plt.show()

print(
    "PCA variance explained:",
    np.round(pca.explained_variance_ratio_ * 100, 2),
    "%"
)

# ------------------------------------------------------------
# 13. Display first-layer learned weights
# ------------------------------------------------------------

weights = pretrained_model.get_layer("hidden_1").get_weights()[0]

plt.figure(figsize=(10, 5))
```

```
for i in range(20):
    plt.subplot(4, 5, i + 1)
    plt.imshow(
        weights[:, i].reshape(28, 28),
        cmap="gray"
    )
    plt.axis("off")

plt.suptitle("First 20 Learned Hidden-Layer Weight Patterns")
plt.tight_layout()
plt.show()
```

Output:

```
Training data shape: (60000, 784)
Test data shape: (10000, 784)

Pretraining first RBM...
RBM Epoch 1/3 - Reconstruction MSE: 0.0748
RBM Epoch 2/3 - Reconstruction MSE: 0.0653
RBM Epoch 3/3 - Reconstruction MSE: 0.0624
First hidden representation shape: (5000, 500)

Pretraining second RBM...
RBM Epoch 1/3 - Reconstruction MSE: 0.0094
RBM Epoch 2/3 - Reconstruction MSE: 0.0082
RBM Epoch 3/3 - Reconstruction MSE: 0.0077
Second RBM pretraining completed.

Fine-tuning pretrained model...
Epoch 1/5
15/15 ━━━━━━━━━━━━━━━━━━━━ 4s 64ms/step - accuracy: 0.1878 - loss: 2.2302 - val_accuracy: 0.2800 - val_loss: 2.0584
Epoch 2/5
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.4178 - loss: 1.8326 - val_accuracy: 0.5500 - val_loss: 1.6162
Epoch 3/5
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.6467 - loss: 1.3728 - val_accuracy: 0.6900 - val_loss: 1.2411
Epoch 4/5
15/15 ━━━━━━━━━━━━━━━━━━━━ 1s 28ms/step - accuracy: 0.7456 - loss: 1.0396 - val_accuracy: 0.7200 - val_loss: 1.0034
Epoch 5/5
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.8022 - loss: 0.8227 - val_accuracy: 0.7400 - val_loss: 0.8534

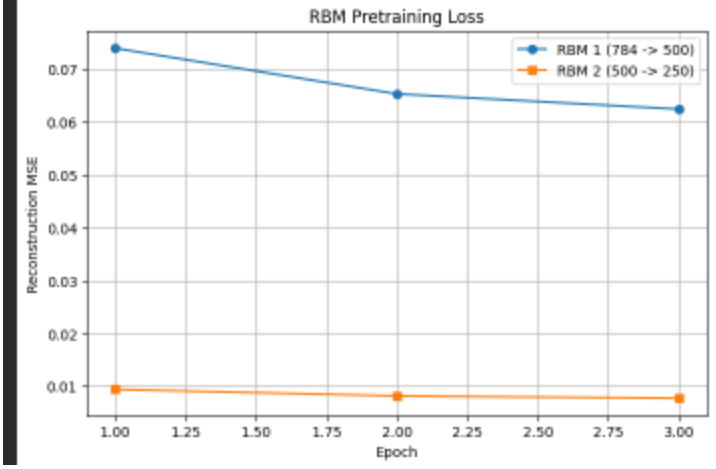
Training randomly initialized model...
Epoch 1/5
15/15 ━━━━━━━━━━━━━━━━━━━━ 3s 56ms/step - accuracy: 0.1633 - loss: 2.2294 - val_accuracy: 0.1900 - val_loss: 2.0011
Epoch 2/5
15/15 ━━━━━━━━━━━━━━━━━━━━ 1s 24ms/step - accuracy: 0.5011 - loss: 1.8123 - val_accuracy: 0.6000 - val_loss: 1.6195
Epoch 3/5
15/15 ━━━━━━━━━━━━━━━━━━━━ 1s 26ms/step - accuracy: 0.7444 - loss: 1.3862 - val_accuracy: 0.7700 - val_loss: 1.2198
Epoch 4/5
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.7922 - loss: 1.0271 - val_accuracy: 0.7700 - val_loss: 0.9523
Epoch 5/5
15/15 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.8222 - loss: 0.7941 - val_accuracy: 0.7700 - val_loss: 0.7986

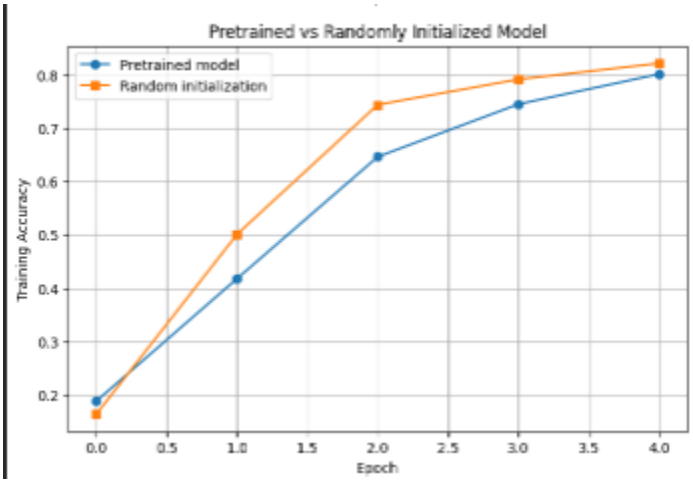
========== MODEL COMPARISON ==========

Pretrained model:
Test loss: 0.8413
Test accuracy: 76.00%
Fine-tuning time: 6.15 seconds

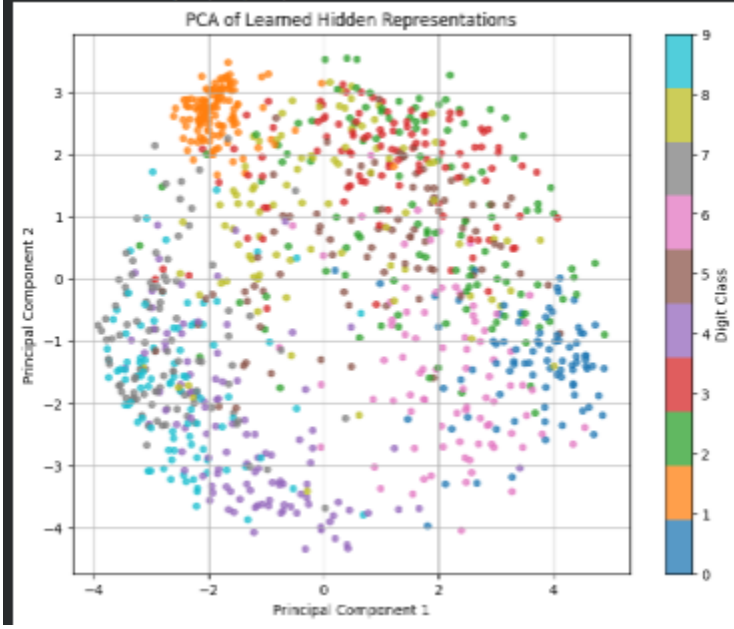
Randomly initialized model:
Test loss: 0.8074
Test accuracy: 79.32%
Training time: 6.11 seconds
```

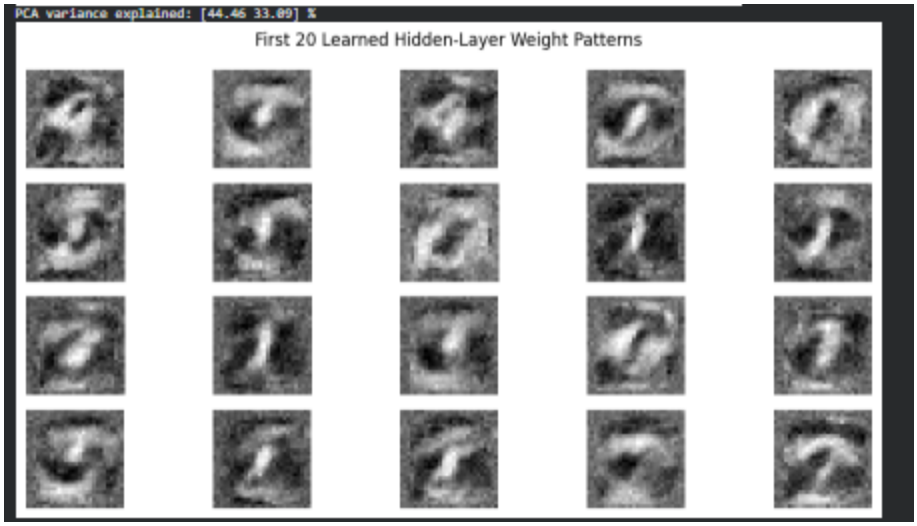
```
RBM pretraining times:
First RBM: 6.17 seconds
Second RBM: 2.96 seconds
```






```
Extracted feature shape: (1000, 250)
```





Experiment 28: Deep Boltzmann Machine / Convolutional Boltzmann Machine Simulation
Problem Statement: A textile company wants to model and generate fabric texture patterns. Spatial structure matters, so a fully connected RBM is not enough. Small grayscale texture patches or MNIST images. The objective is to simulate a two-layer DBM or a convolutional RBM and study its representations. Use NumPy, SciPy, TensorFlow, Keras, Matplotlib.
Tasks:
1. Build a two-hidden-layer DBM with mean-field inference, or a convolutional RBM with shared filters.
2. Train with approximate learning (mean-field plus Gibbs sampling).
3. Visualise filters and hidden-unit activations.
4. Generate samples and reconstructions.
5. Compare with the single-layer RBM from Experiment 25 on reconstruction quality.

Formulas:

1. DBM energy function

$$E(v, h^{(1)}, h^{(2)}) = -\, v^T W_1 h^{(1)} - (h^{(1)})^T W_2 h^{(2)} - a^T v - b_1^T h^{(1)} - b_2^T h^{(2)}$$

2. Mean-field approximation

$$\mu_i^{(1)} = \sigma\left(b_i^{(1)} + \sum_j W_{1,ji} v_j + \sum_k W_{2,ik} \mu_k^{(2)}\right)$$

3. Reconstruction

$$\hat{v} = \sigma(W_1 \mu^{(1)} + a)$$

4. Reconstruction MSE

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (v_i - \hat{v}_i)^2$$

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist

np.random.seed(42)

(x_train, _), (x_test, _) = mnist.load_data()
x_train = (x_train[:4000].reshape(-1, 784) / 255.0 > 0.5).astype("float32")
x_test = (x_test[:200].reshape(-1, 784) / 255.0 > 0.5).astype("float32")

def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -30, 30)))

# Two-hidden-layer DBM: 784 -> 128 -> 64
W1 = np.random.normal(0, 0.01, (784, 128)).astype("float32")
W2 = np.random.normal(0, 0.01, (128, 64)).astype("float32")
a = np.zeros(784, dtype="float32")
b1 = np.zeros(128, dtype="float32")
b2 = np.zeros(64, dtype="float32")

def mean_field(v, iterations=5):
    # Approximate posterior hidden activations
    h1 = sigmoid(v @ W1 + b1)
    h2 = sigmoid(h1 @ W2 + b2)

    for _ in range(iterations):
        h1 = sigmoid(v @ W1 + h2 @ W2.T + b1)
        h2 = sigmoid(h1 @ W2 + b2)

    return h1, h2
```

```
def reconstruct(v):
    h1, h2 = mean_field(v)
    return sigmoid(h1 @ W1.T + a), h1, h2

# Approximate contrastive training
EPOCHS = 5
BATCH = 64
LR = 0.005
errors = []

for epoch in range(EPOCHS):
    order = np.random.permutation(len(x_train))
    epoch_errors = []

    for start in range(0, len(x_train), BATCH):
        v0 = x_train[order[start:start+BATCH]]
        if len(v0) < 2:
            continue

        h10, h20 = mean_field(v0)

        # One approximate Gibbs reconstruction step
        pv = sigmoid(h10 @ W1.T + a)
        vk = (np.random.rand(*pv.shape) < pv).astype("float32")
        h1k, h2k = mean_field(vk)

        # Approximate positive/negative association updates
        W1 += LR * (v0.T @ h10 - vk.T @ h1k) / len(v0)
        W2 += LR * (h10.T @ h20 - h1k.T @ h2k) / len(v0)
        a += LR * np.mean(v0 - vk, axis=0)
        b1 += LR * np.mean(h10 - h1k, axis=0)
        b2 += LR * np.mean(h20 - h2k, axis=0)

        epoch_errors.append(np.mean((v0 - pv) ** 2))

    errors.append(np.mean(epoch_errors))
    print(f"Epoch {epoch+1}: reconstruction MSE={errors[-1]:.4f}")

# Plot training reconstruction error
plt.figure(figsize=(7, 4))
plt.plot(range(1, EPOCHS+1), errors, marker="o")
plt.title("DBM Simulation Reconstruction Error")
plt.xlabel("Epoch")
plt.ylabel("MSE")
```

```
plt.grid(True)
plt.show()

# Visualize first-layer weights
fig, axes = plt.subplots(4, 8, figsize=(10, 5))
for i, ax in enumerate(axes.flat):
    ax.imshow(W1[:, i].reshape(28, 28), cmap="gray")
    ax.axis("off")
plt.suptitle("First-Layer Learned Weight Patterns")
plt.tight_layout()
plt.show()

# Visualize hidden activations
_, h1, h2 = reconstruct(x_test[:10])
fig, axes = plt.subplots(2, 10, figsize=(12, 3))
for i in range(10):
    axes[0, i].imshow(h1[i].reshape(8, 16), cmap="viridis")
    axes[0, i].axis("off")
    axes[1, i].imshow(h2[i].reshape(8, 8), cmap="viridis")
    axes[1, i].axis("off")
axes[0, 0].set_ylabel("Hidden 1")
axes[1, 0].set_ylabel("Hidden 2")
plt.suptitle("Mean-Field Hidden Activations")
plt.tight_layout()
plt.show()

# Reconstruct test images
recon, _, _ = reconstruct(x_test[:10])

fig, axes = plt.subplots(2, 10, figsize=(12, 3))
for i in range(10):
    axes[0, i].imshow(x_test[i].reshape(28, 28), cmap="gray")
    axes[0, i].axis("off")
    axes[1, i].imshow(recon[i].reshape(28, 28), cmap="gray")
    axes[1, i].axis("off")
axes[0, 0].set_ylabel("Original")
axes[1, 0].set_ylabel("Reconstructed")
plt.tight_layout()
plt.show()

dbm_mse = np.mean((x_test[:10] - recon) ** 2)
print("DBM simulation reconstruction MSE:", round(float(dbm_mse), 4))

# Single-layer RBM baseline with a comparable hidden dimension
```

```
W_rbm = np.random.normal(0, 0.01, (784, 128)).astype("float32")
vb_rbm = np.zeros(784, dtype="float32")
hb_rbm = np.zeros(128, dtype="float32")

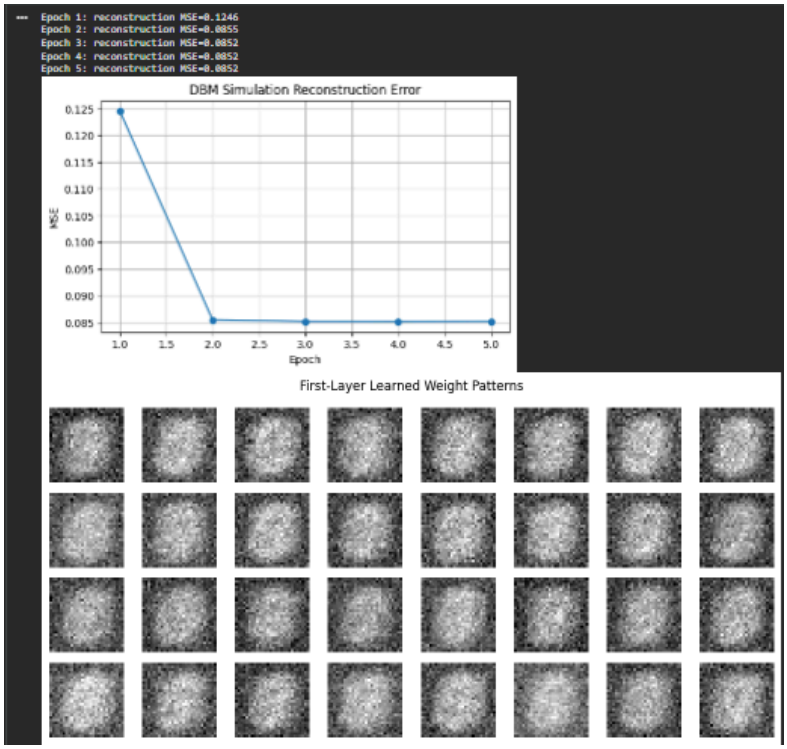
for epoch in range(EPOCHS):
    order = np.random.permutation(len(x_train))
    for start in range(0, len(x_train), BATCH):
        v0 = x_train[order[start:start+BATCH]]
        ph0 = sigmoid(v0 @ W_rbm + hb_rbm)
        h0 = (np.random.rand(*ph0.shape) < ph0).astype("float32")
        pv1 = sigmoid(h0 @ W_rbm.T + vb_rbm)
        v1 = (np.random.rand(*pv1.shape) < pv1).astype("float32")
        ph1 = sigmoid(v1 @ W_rbm + hb_rbm)

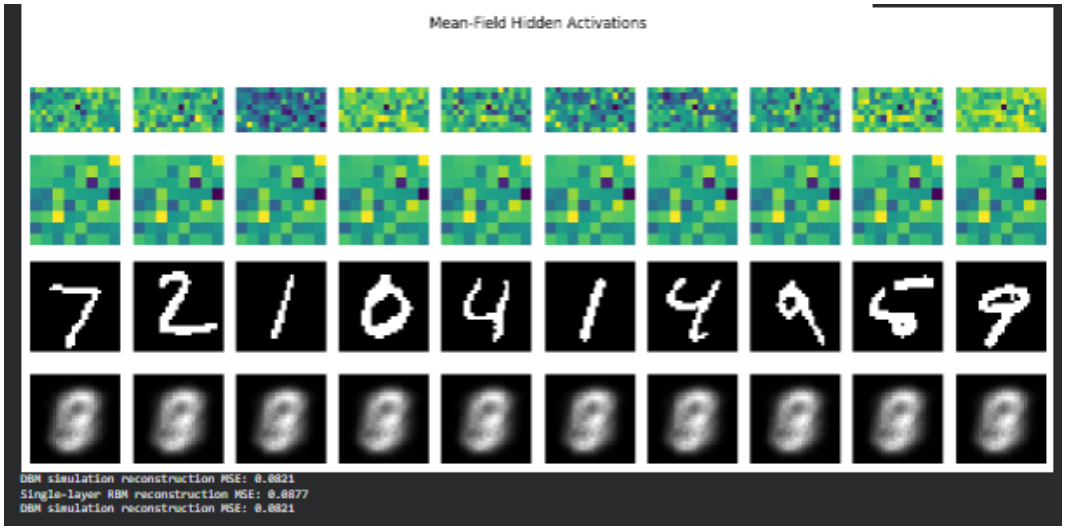
        W_rbm += LR * (v0.T @ ph0 - v1.T @ ph1) / len(v0)
        vb_rbm += LR * np.mean(v0 - v1, axis=0)
        hb_rbm += LR * np.mean(ph0 - ph1, axis=0)

rbm_recon = sigmoid(sigmoid(x_test[:10] @ W_rbm + hb_rbm) @ W_rbm.T + vb_rbm)
rbm_mse = np.mean((x_test[:10] - rbm_recon) ** 2)

print("Single-layer RBM reconstruction MSE:", round(float(rbm_mse), 4))
print("DBM simulation reconstruction MSE:", round(float(dbm_mse), 4))
```

Output:





Experiment 29: VAE/GAN Sample Generation
Problem Statement: A fashion retailer wants to generate new product images to prototype designs before manufacturing. The Fashion-MNIST dataset (clothing and footwear images, 28×28). The objective is to build a VAE and a GAN and generate new samples. Use TensorFlow, Keras, NumPy, Matplotlib.
Tasks:
1. Build a VAE with encoder, reparameterisation and decoder; train with reconstruction plus KL loss.
2. Generate images by sampling the latent space; plot a 2-D latent-space grid.
3. Build a DCGAN with a generator and a discriminator.
4. Train the GAN and save generated images every few epochs.
5. Plot generator and discriminator loss curves.

Formulas:

Part A: Variational Autoencoder

1. Reparameterization

$$z = \mu + \sigma \odot \epsilon$$

2. KL divergence

$$D_{KL} = -\frac{1}{2}\sum_{j}(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$$

3. VAE loss

$$L_{VAE} = L_{reconstruction} + D_{KL}$$

Part B: Generative Adversarial Network

4. Generator

$$\hat{x} = G(z)$$

5. Discriminator

$$D(x) = P(real|x)$$

6. GAN minimax objective

$$\min_G \max_D V(D, G)$$

$$V(D, G) = E_{x \sim p_{data}}[\log D(x)] + E_{z \sim p_z}[\log(1 - D(G(z)))]$$

7. Generator non-saturating loss

$$L_G = - E_{z \sim p_z}[\log D(G(z))]$$

8. Discriminator loss

$$L_D = - E_x[\log D(x)] - E_z[\log(1 - D(G(z)))]$$

Code:

```
import os
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras import layers, Model
```

```
np.random.seed(42)
tf.random.set_seed(42)
os.makedirs("fashion_generated", exist_ok=True)

# 1. Load Fashion-MNIST
(x_train, _), (x_test, _) = fashion_mnist.load_data()
x_train = x_train.astype("float32") / 255.0
x_train = x_train[..., np.newaxis]
x_train = x_train[:12000]

BATCH = 128
LATENT_DIM = 2
EPOCHS_VAE = 5
EPOCHS_GAN = 5

train_ds = tf.data.Dataset.from_tensor_slices(x_train)
train_ds = train_ds.shuffle(5000).batch(BATCH).prefetch(tf.data.AUTOTUNE)

# ------------------------------------------------------------
# 2. Variational Autoencoder (VAE)
# ------------------------------------------------------------
latent_dim = LATENT_DIM

encoder_input = layers.Input(shape=(28, 28, 1))
x = layers.Flatten()(encoder_input)
x = layers.Dense(256, activation="relu")(x)
z_mean = layers.Dense(latent_dim, name="z_mean")(x)
z_log_var = layers.Dense(latent_dim, name="z_log_var")(x)

def sample_latent(args):
    mean, log_var = args
    eps = tf.random.normal(shape=tf.shape(mean))
    return mean + tf.exp(0.5 * log_var) * eps

z = layers.Lambda(sample_latent)([z_mean, z_log_var])
encoder = Model(encoder_input, [z_mean, z_log_var, z], name="encoder")

decoder_input = layers.Input(shape=(latent_dim,))
x = layers.Dense(256, activation="relu")(decoder_input)
x = layers.Dense(784, activation="sigmoid")(x)
decoder_output = layers.Reshape((28, 28, 1))(x)
decoder = Model(decoder_input, decoder_output, name="decoder")
```

```
class VAE(Model):
    def __init__(self, encoder, decoder):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.loss_tracker = tf.keras.metrics.Mean(name="loss")
        self.recon_tracker = tf.keras.metrics.Mean(name="reconstruction_loss")
        self.kl_tracker = tf.keras.metrics.Mean(name="kl_loss")

    @property
    def metrics(self):
        return [self.loss_tracker, self.recon_tracker, self.kl_tracker]

    def train_step(self, data):
        if isinstance(data, tuple):
            data = data[0]

        with tf.GradientTape() as tape:
            mean, log_var, z = self.encoder(data, training=True)
            reconstruction = self.decoder(z, training=True)

            # Binary cross-entropy summed across pixels
            bce = tf.keras.backend.binary_crossentropy(data, reconstruction)
            recon_loss = tf.reduce_mean(tf.reduce_sum(bce, axis=[1, 2]))

            kl_loss = -0.5 * tf.reduce_mean(tf.reduce_sum(
                1 + log_var - tf.square(mean) - tf.exp(log_var), axis=1
            ))
            loss = recon_loss + kl_loss

        grads = tape.gradient(loss, self.trainable_variables)
        self.optimizer.apply_gradients(zip(grads, self.trainable_variables))

        self.loss_tracker.update_state(loss)
        self.recon_tracker.update_state(recon_loss)
        self.kl_tracker.update_state(kl_loss)

        return {m.name: m.result() for m in self.metrics}

vae = VAE(encoder, decoder)
vae.compile(optimizer=tf.keras.optimizers.Adam())

vae_history = vae.fit(train_ds, epochs=EPOCHS_VAE, verbose=1)
```

```
# Generate samples from a 2-D latent-space grid
grid_size = 10
grid_x = np.linspace(-2.5, 2.5, grid_size)
grid_y = np.linspace(-2.5, 2.5, grid_size)
latent_points = np.array([
    [x, y] for y in grid_y[::-1] for x in grid_x
], dtype="float32")

vae_grid_images = decoder.predict(latent_points, verbose=0)

fig, axes = plt.subplots(grid_size, grid_size, figsize=(10, 10))
for i, ax in enumerate(axes.flat):
    ax.imshow(vae_grid_images[i].squeeze(), cmap="gray")
    ax.axis("off")
plt.suptitle("VAE 2-D Latent-Space Grid")
plt.tight_layout()
plt.show()

# Save VAE-generated samples
vae_samples = decoder.predict(
    np.random.normal(size=(100, latent_dim)).astype("float32"),
    verbose=0
)
for i in range(0, 100, 25):
    fig, axes = plt.subplots(5, 5, figsize=(5, 5))
    for j, ax in enumerate(axes.flat):
        ax.imshow(vae_samples[i+j].squeeze(), cmap="gray")
        ax.axis("off")
    plt.tight_layout()
    plt.savefig(f"fashion_generated/vae_epoch_final_{i}.png")
    plt.show()

# ---------------------------------------------------------
# 3. Deep Convolutional GAN (DCGAN)
# ---------------------------------------------------------
noise_dim = 100

generator = tf.keras.Sequential([
    layers.Input(shape=(noise_dim,)),
    layers.Dense(7 * 7 * 128, use_bias=False),
    layers.BatchNormalization(),
    layers.LeakyReLU(),
    layers.Reshape((7, 7, 128)),
    layers.Conv2DTranspose(64, 4, strides=2, padding="same", use_bias=False),
```

```
    layers.BatchNormalization(),
    layers.LeakyReLU(),
    layers.Conv2DTranspose(1, 4, strides=2, padding="same", activation="sigmoid")
], name="generator")

discriminator = tf.keras.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(64, 4, strides=2, padding="same"),
    layers.LeakyReLU(),
    layers.Dropout(0.3),
    layers.Conv2D(128, 4, strides=2, padding="same"),
    layers.LeakyReLU(),
    layers.Flatten(),
    layers.Dense(1)
], name="discriminator")

bce = tf.keras.losses.BinaryCrossentropy(from_logits=True)
g_optimizer = tf.keras.optimizers.Adam(1e-4, beta_1=0.5)
d_optimizer = tf.keras.optimizers.Adam(1e-4, beta_1=0.5)

def gan_train_step(real_images):
    batch_size = tf.shape(real_images)[0]
    noise = tf.random.normal((batch_size, noise_dim))

    with tf.GradientTape() as g_tape, tf.GradientTape() as d_tape:
        fake_images = generator(noise, training=True)
        real_logits = discriminator(real_images, training=True)
        fake_logits = discriminator(fake_images, training=True)

        g_loss = bce(tf.ones_like(fake_logits), fake_logits)
        d_loss_real = bce(tf.ones_like(real_logits), real_logits)
        d_loss_fake = bce(tf.zeros_like(fake_logits), fake_logits)
        d_loss = d_loss_real + d_loss_fake

    g_grads = g_tape.gradient(g_loss, generator.trainable_variables)
    d_grads = d_tape.gradient(d_loss, discriminator.trainable_variables)

    g_optimizer.apply_gradients(zip(g_grads, generator.trainable_variables))
    d_optimizer.apply_gradients(zip(d_grads, discriminator.trainable_variables))

    return float(g_loss.numpy()), float(d_loss.numpy())

g_losses, d_losses = [], []
```

```
def save_gan_samples(epoch):
    noise = tf.random.normal((25, noise_dim))
    generated = generator(noise, training=False).numpy()

    fig, axes = plt.subplots(5, 5, figsize=(5, 5))
    for i, ax in enumerate(axes.flat):
        ax.imshow(generated[i].squeeze(), cmap="gray")
        ax.axis("off")
    plt.suptitle(f"GAN Generated Samples - Epoch {epoch}")
    plt.tight_layout()
    plt.savefig(f"fashion_generated/gan_epoch_{epoch}.png")
    plt.show()

for epoch in range(1, EPOCHS_GAN + 1):
    epoch_g, epoch_d = [], []

    for batch in train_ds:
        gl, dl = gan_train_step(batch)
        epoch_g.append(gl)
        epoch_d.append(dl)

    g_losses.append(np.mean(epoch_g))
    d_losses.append(np.mean(epoch_d))

    print(
        f"GAN Epoch {epoch}/{EPOCHS_GAN} | "
        f"G loss={g_losses[-1]:.4f} | D loss={d_losses[-1]:.4f}"
    )

    # Save images every epoch
    save_gan_samples(epoch)

# 4. Plot VAE losses
plt.figure(figsize=(8, 4))
plt.plot(vae_history.history["loss"], label="Total VAE loss")
plt.plot(vae_history.history["reconstruction_loss"], label="Reconstruction loss")
plt.plot(vae_history.history["kl_loss"], label="KL loss")
plt.title("VAE Training Losses")
plt.xlabel("Epoch")
plt.legend()
plt.grid(True)
plt.show()

# Plot GAN losses
```

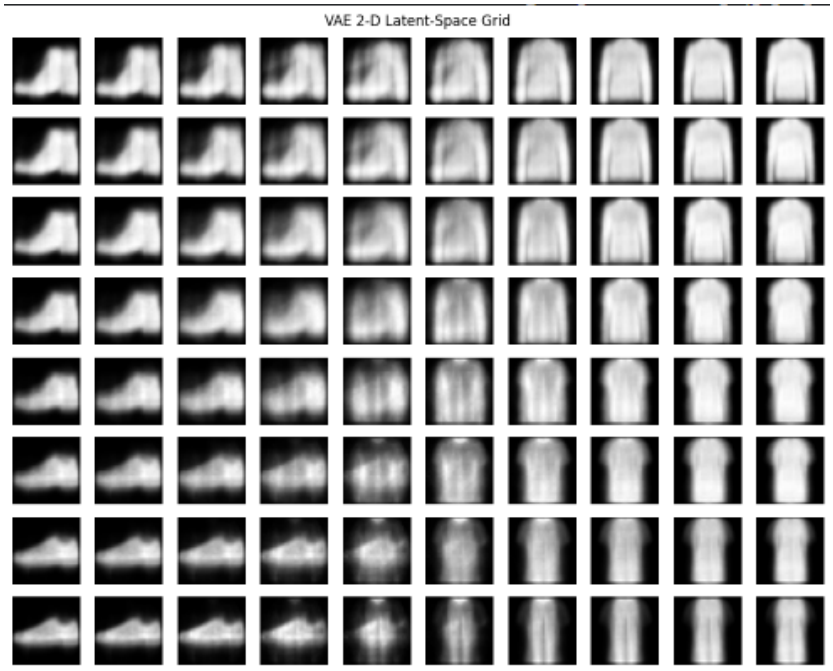
```
plt.figure(figsize=(8, 4))
plt.plot(g_losses, marker="o", label="Generator loss")
plt.plot(d_losses, marker="o", label="Discriminator loss")
plt.title("DCGAN Training Losses")
plt.xlabel("Epoch")
plt.legend()
plt.grid(True)
plt.show()

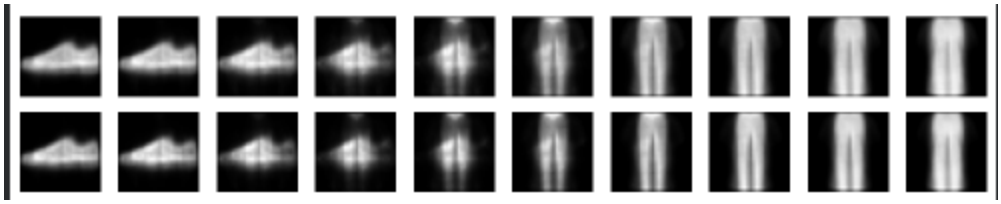
# Save trained models for Experiment 30
encoder.save("fashion_vae_encoder.keras")
decoder.save("fashion_vae_decoder.keras")
generator.save("fashion_gan_generator.keras")

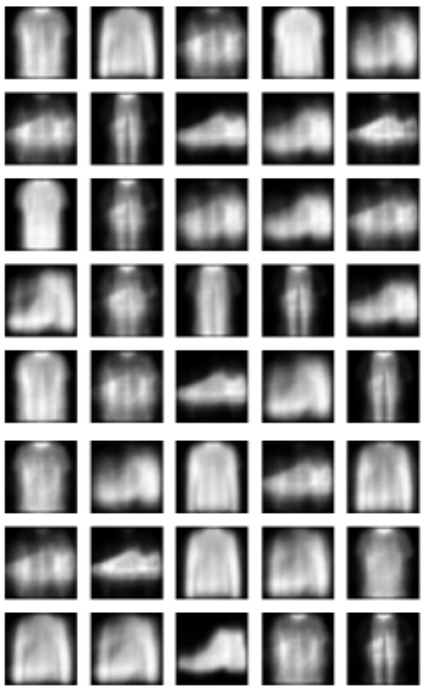
print("Saved VAE encoder, decoder and GAN generator.")
```

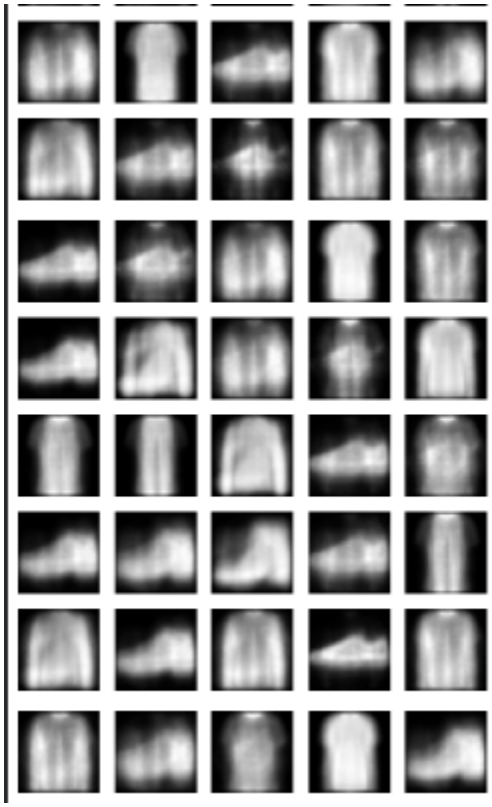
Output:

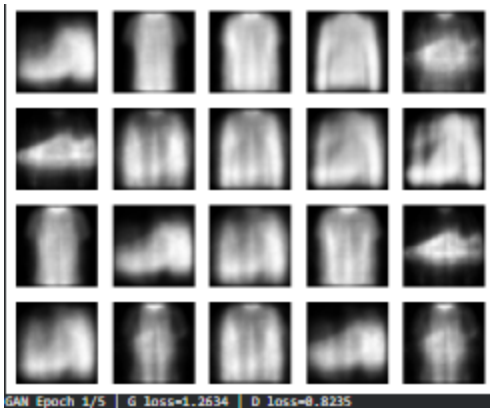
```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 2us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Epoch 1/5
94/94 ━━━━━━━━━━━━━━━━━━━━ 8s 35ms/step - kl_loss: 15.7748 - loss: 377.4819 - reconstruction_loss: 361.7072
Epoch 2/5
94/94 ━━━━━━━━━━━━━━━━━━━━ 4s 25ms/step - kl_loss: 8.9617 - loss: 299.1573 - reconstruction_loss: 290.1956
Epoch 3/5
94/94 ━━━━━━━━━━━━━━━━━━━━ 2s 25ms/step - kl_loss: 7.3985 - loss: 289.9511 - reconstruction_loss: 282.5527
Epoch 4/5
94/94 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - kl_loss: 7.0214 - loss: 285.8979 - reconstruction_loss: 278.8765
Epoch 5/5
94/94 ━━━━━━━━━━━━━━━━━━━━ 3s 28ms/step - kl_loss: 6.7878 - loss: 283.2379 - reconstruction_loss: 276.4501
```



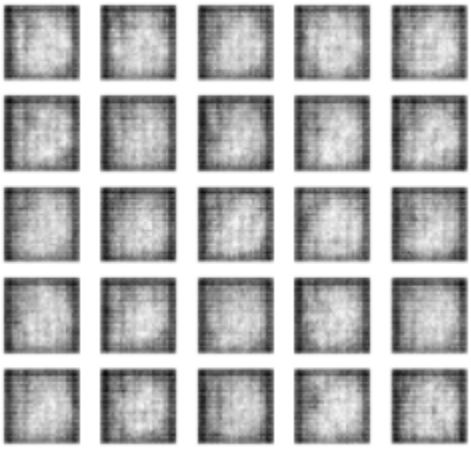


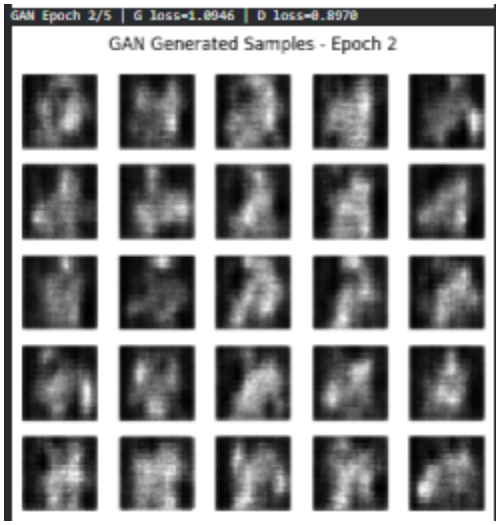




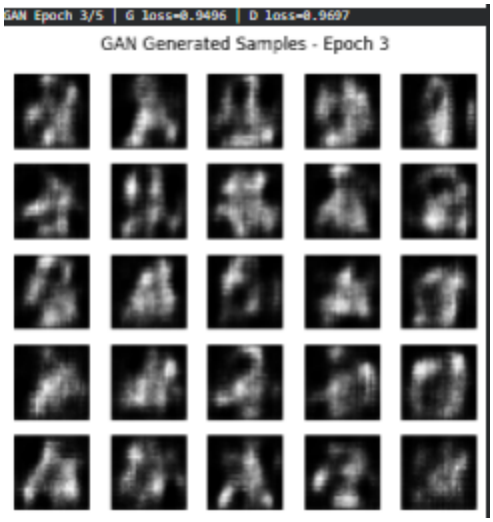


GAN Generated Samples - Epoch 1





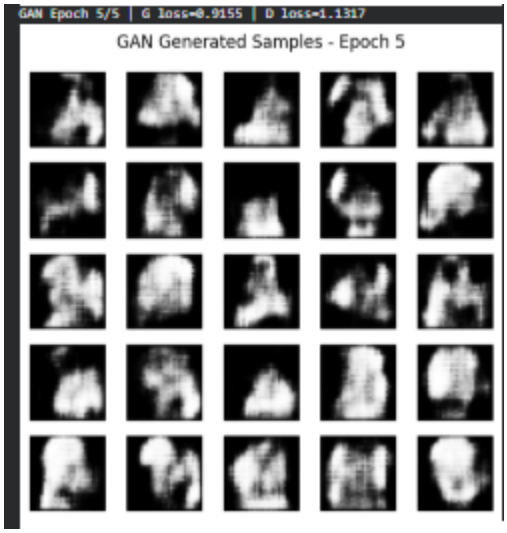
GAN Epoch 3/5 | G loss=0.9496 | D loss=0.9697

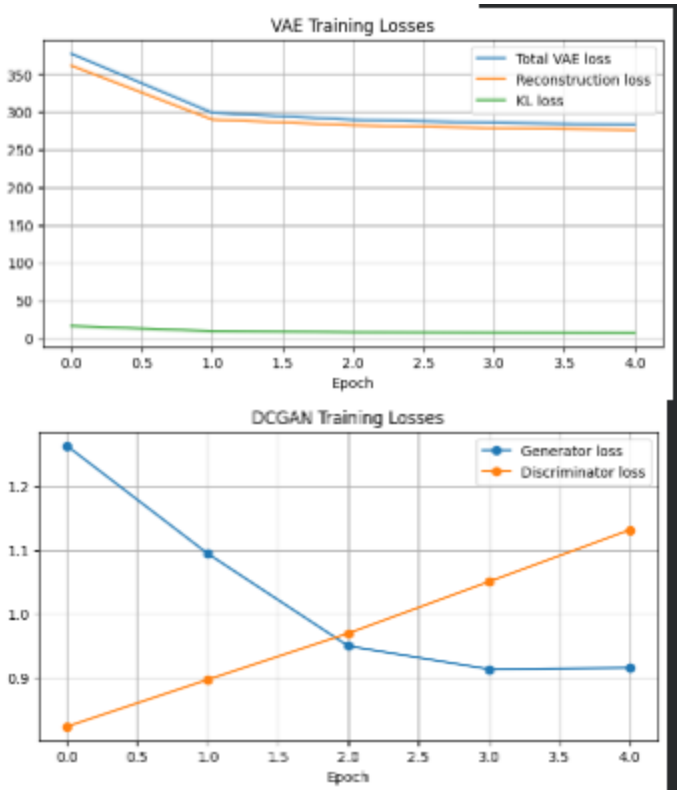


GAN Epoch 4/5 | G loss=0.9134 | D loss=1.0506



GAN Epoch 5/5 | G loss=0.9155 | D loss=1.1317





Experiment 30: Generative Model Output Comparison and Evaluation
Problem Statement: The retailer's management must choose one generative model for production. They need objective evidence of which produces better and more varied images. The VAE and GAN outputs from Experiment 29 on Fashion-MNIST. The objective is to evaluate and compare generative models with quantitative and qualitative measures. Use TensorFlow, Keras, NumPy, Pandas, Scikit-learn, Matplotlib.
Tasks:
1. Generate 1,000 samples from each model.
2. Compute a classifier-based Inception Score and FID using a classifier trained on Fashion-MNIST.
3. Measure diversity and check for mode collapse through the class distribution of samples.
4. Compare real and generated feature distributions with t-SNE or PCA plots.
5. Tabulate all metrics with Pandas and write a recommendation.

Formulas:

1. Inception Score (IS)

$$IS = \exp\left(E_x\left[D_{KL}(p(y|x)||p(y))\right]\right)$$

2. Class entropy

$$H(Y) = -\sum_{i=1}^{C} p_i \log(p_i)$$

3. Mean pairwise pixel distance

$$D = \frac{2}{n(n-1)} \sum_{i<j} d(x_i, x_j)$$

4. Classification accuracy

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \times 100$$

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras import layers, Model
from sklearn.decomposition import PCA
from scipy.linalg import sqrtm

np.random.seed(42)
tf.random.set_seed(42)

# 1. Load Fashion-MNIST and the saved generators from Experiment 29
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

x_train = x_train[..., np.newaxis]
x_test = x_test[..., np.newaxis]

decoder = tf.keras.models.load_model(
    "fashion_vae_decoder.keras", compile=False
)
generator = tf.keras.models.load_model(
    "fashion_gan_generator.keras", compile=False
```

```
)

# 2. Train a Fashion-MNIST classifier for evaluation
classifier_input = layers.Input(shape=(28, 28, 1))
x = layers.Conv2D(32, 3, activation="relu", padding="same")(classifier_input)
x = layers.MaxPooling2D()(x)
x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)
x = layers.MaxPooling2D()(x)
x = layers.Flatten()(x)
feature_layer = layers.Dense(128, activation="relu", name="features")(x)
classifier_output = layers.Dense(10, activation="softmax")(feature_layer)

classifier = Model(classifier_input, classifier_output)
classifier.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

classifier.fit(
    x_train[:20000], y_train[:20000],
    epochs=5,
    batch_size=128,
    validation_split=0.1,
    verbose=1
)

test_loss, test_accuracy = classifier.evaluate(
    x_test, y_test, verbose=0
)
print("Classifier test accuracy:", round(test_accuracy, 4))

feature_extractor = Model(
    classifier.input,
    classifier.get_layer("features").output
)

# 3. Generate 1,000 samples from each model
N = 1000

vae_z = np.random.normal(size=(N, 2)).astype("float32")
vae_images = decoder.predict(vae_z, verbose=0)

gan_z = np.random.normal(size=(N, 100)).astype("float32")
```

```
gan_images = generator.predict(gan_z, verbose=0)

real_images = x_test[:N]

# 4. Classifier-based Inception Score
def classifier_inception_score(images, batch_size=128, splits=5):
    probs = classifier.predict(images, batch_size=batch_size, verbose=0)
    scores = []

    for part in np.array_split(probs, splits):
        if len(part) == 0:
            continue

        marginal = np.mean(part, axis=0, keepdims=True)
        kl = part * (
            np.log(np.maximum(part, 1e-10))
            - np.log(np.maximum(marginal, 1e-10))
        )
        kl_sum = np.sum(kl, axis=1)
        scores.append(np.exp(np.mean(kl_sum)))

    return float(np.mean(scores)), float(np.std(scores))

# 5. FID-like distance using the classifier's learned feature space
def feature_fid(real, generated):
    real_features = feature_extractor.predict(
        real, batch_size=128, verbose=0
    )
    gen_features = feature_extractor.predict(
        generated, batch_size=128, verbose=0
    )

    mu_real = np.mean(real_features, axis=0)
    mu_gen = np.mean(gen_features, axis=0)

    cov_real = np.cov(real_features, rowvar=False)
    cov_gen = np.cov(gen_features, rowvar=False)

    covmean = sqrtm(cov_real @ cov_gen)

    # Numerical rounding can produce a tiny imaginary component
    if np.iscomplexobj(covmean):
        covmean = covmean.real
```

```
    diff = mu_real - mu_gen
    fid = diff @ diff + np.trace(cov_real + cov_gen - 2 * covmean)

    return float(max(fid, 0.0)), real_features, gen_features

vae_is, vae_is_std = classifier_inception_score(vae_images)
gan_is, gan_is_std = classifier_inception_score(gan_images)

vae_fid, real_features, vae_features = feature_fid(real_images, vae_images)
gan_fid, _, gan_features = feature_fid(real_images, gan_images)

# 6. Diversity and class distribution
def get_class_statistics(images):
    probs = classifier.predict(images, batch_size=128, verbose=0)
    predicted_classes = np.argmax(probs, axis=1)

    counts = np.bincount(predicted_classes, minlength=10)
    proportions = counts / counts.sum()

    # Entropy of predicted class distribution
    entropy = -np.sum(
        proportions * np.log(np.maximum(proportions, 1e-10))
    )

    # Average pairwise pixel distance, estimated using a subset
    flat = images[:200].reshape(min(200, len(images)), -1)
    distances = []

    for i in range(len(flat)):
        for j in range(i + 1, len(flat)):
            distances.append(np.mean(np.abs(flat[i] - flat[j])))

    mean_pixel_distance = float(np.mean(distances)) if distances else 0.0

    return proportions, entropy, mean_pixel_distance

vae_classes, vae_entropy, vae_diversity = get_class_statistics(vae_images)
gan_classes, gan_entropy, gan_diversity = get_class_statistics(gan_images)
real_classes, real_entropy, real_diversity = get_class_statistics(real_images)

# 7. Tabulate metrics
metrics_df = pd.DataFrame([
    {
        "Model": "VAE",
```

```
        "Classifier IS": vae_is,
        "IS Std": vae_is_std,
        "Feature FID-like (lower better)": vae_fid,
        "Class Entropy": vae_entropy,
        "Mean Pixel Distance": vae_diversity
    },
    {
        "Model": "GAN",
        "Classifier IS": gan_is,
        "IS Std": gan_is_std,
        "Feature FID-like (lower better)": gan_fid,
        "Class Entropy": gan_entropy,
        "Mean Pixel Distance": gan_diversity
    },
    {
        "Model": "Real test images",
        "Classifier IS": classifier_inception_score(real_images)[0],
        "IS Std": classifier_inception_score(real_images)[1],
        "Feature FID-like (lower better)": 0.0,
        "Class Entropy": real_entropy,
        "Mean Pixel Distance": real_diversity
    }
])

display(metrics_df.round(4))

# 8. Compare predicted class distributions
class_names = [
    "T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"
]

distribution_df = pd.DataFrame({
    "Class": class_names,
    "Real": real_classes,
    "VAE": vae_classes,
    "GAN": gan_classes
})

display(distribution_df.round(3))

x = np.arange(10)
width = 0.25
```

```
plt.figure(figsize=(13, 5))
plt.bar(x - width, real_classes, width, label="Real")
plt.bar(x, vae_classes, width, label="VAE")
plt.bar(x + width, gan_classes, width, label="GAN")
plt.xticks(x, class_names, rotation=45, ha="right")
plt.ylabel("Predicted class proportion")
plt.title("Class Distribution: Real vs Generated Samples")
plt.legend()
plt.tight_layout()
plt.show()

# 9. PCA visualization of real and generated classifier features
# Fit PCA on combined features so all groups share the same axes
combined_features = np.vstack([
    real_features, vae_features, gan_features
])

features_2d = PCA(n_components=2).fit_transform(combined_features)

real_2d = features_2d[:N]
vae_2d = features_2d[N:2*N]
gan_2d = features_2d[2*N:]

plt.figure(figsize=(9, 7))
plt.scatter(
    real_2d[:, 0], real_2d[:, 1],
    s=10, alpha=0.3, label="Real"
)
plt.scatter(
    vae_2d[:, 0], vae_2d[:, 1],
    s=10, alpha=0.3, label="VAE"
)
plt.scatter(
    gan_2d[:, 0], gan_2d[:, 1],
    s=10, alpha=0.3, label="GAN"
)
plt.title("PCA of Real and Generated Image Features")
plt.xlabel("PCA 1")
plt.ylabel("PCA 2")
plt.legend()
plt.show()

# 10. Display generated samples
fig, axes = plt.subplots(2, 10, figsize=(12, 3))
```

```
for i in range(10):
    axes[0, i].imshow(vae_images[i].squeeze(), cmap="gray")
    axes[0, i].axis("off")

    axes[1, i].imshow(gan_images[i].squeeze(), cmap="gray")
    axes[1, i].axis("off")

axes[0, 0].set_ylabel("VAE")
axes[1, 0].set_ylabel("GAN")
plt.suptitle("Generated Fashion-MNIST Samples")
plt.tight_layout()
plt.show()
```

Output:

```
Epoch 1/5
141/141 ━━━━━━━━━━━━━━━━━━━━ 39s 259ms/step - accuracy: 0.7568 - loss: 0.6881 - val_accuracy: 0.8405 - val_loss: 0.4501
Epoch 2/5
141/141 ━━━━━━━━━━━━━━━━━━━━ 25s 178ms/step - accuracy: 0.8521 - loss: 0.4101 - val_accuracy: 0.8720 - val_loss: 0.3663
Epoch 3/5
141/141 ━━━━━━━━━━━━━━━━━━━━ 36s 143ms/step - accuracy: 0.8706 - loss: 0.3570 - val_accuracy: 0.8750 - val_loss: 0.3405
Epoch 4/5
141/141 ━━━━━━━━━━━━━━━━━━━━ 22s 153ms/step - accuracy: 0.8858 - loss: 0.3176 - val_accuracy: 0.8870 - val_loss: 0.3155
Epoch 5/5
141/141 ━━━━━━━━━━━━━━━━━━━━ 20s 142ms/step - accuracy: 0.8948 - loss: 0.2892 - val_accuracy: 0.8915 - val_loss: 0.3009
Classifier test accuracy: 0.8815
/tmp/ipykernel_4642/530618292.py:110: LinAlgWarning: Matrix is singular. The result might be inaccurate or the array might not have a square root.
  covmean = sqrtm(cov_real @ cov_gen)
```

	Model	Classifier IS	IS Std	Feature FID-like (lower better)	Class Entropy	Mean Pixel Distance
0	VAE	1.7136	0.0367	185.3701	1.7922	0.1676
1	GAN	3.1376	0.0859	52.4063	2.0990	0.2709
2	Real test images	7.2450	0.1074	0.0000	2.2953	0.2822

	Class	Real	VAE	GAN
0	T-shirt/top	0.112	0.263	0.113
1	Trouser	0.104	0.005	0.003
2	Pullover	0.124	0.038	0.019
3	Dress	0.093	0.019	0.111
4	Coat	0.085	0.013	0.121
5	Sandal	0.088	0.000	0.168
6	Shirt	0.114	0.249	0.158
7	Sneaker	0.096	0.104	0.049
8	Bag	0.091	0.160	0.159
9	Ankle boot	0.093	0.149	0.099

